

Technical Report on the Implemented Corrosion Analysis Pipeline

Repository-grounded reconstruction of data processing, feature engineering, benchmarking, diagnostics, and exploratory proxy-RUL outputs

Project root: `main_4`

Date: March 17, 2026

This report is grounded in the repository code, YAML configuration, generated CSV artifacts, plots, diagnostics, and logs.

When earlier prose conflicted with repository outputs, the repository was treated as authoritative.

Abstract

This report documents what the implemented repository actually does for corrosion-image analysis, what it does not do, and what conclusions are technically supported by the generated outputs. The system operates on a longitudinal dataset of 791 aligned image-table observations from 48 specimens, with dense surface-corrosion labels and only 48 terminal structural-label rows. The implemented formulation is not direct image-to-RUL prediction. Instead, the codebase implements a staged pipeline: data audit and alignment, deterministic specimen mapping, interpretable image-feature extraction, stage-specific feature engineering, leakage-safe grouped evaluation, surface-corrosion modelling, hidden-damage modelling, specimen-level degradation fitting, and exploratory threshold-based proxy-RUL status analysis.

The strongest implemented components are the audit and alignment logic, the leakage-safe split design, the interpretable image-feature layer, the temporal progression diagnostics, and the feature-diagnostics layer that exposes redundancy, duplication, and collinearity. The main negative findings are equally important: the surface target `surface_total_rust_pct` is largely reconstructive because it is almost numerically identical to the engineered rust-area feature; the structural targets are sparse and terminal-only; hidden-damage performance degrades sharply under leave-one-campaign-out evaluation; and the final robustness-selected hidden-damage models are metadata-only rather than image-driven. The degradation and proxy-RUL stages are implemented and reproducible, but they remain descriptive or exploratory because they are built on model-predicted hidden damage rather than dense observed structural trajectories and because no future threshold crossings are generated within the current 365-day projection horizon.

Contents

1	Introduction	4
2	Problem Framing	4
2.1	What problem the implemented system actually solves	4
2.2	What is directly supervised and what is not	4
2.3	Why direct image-to-RUL is not supported	4
3	Repository-Grounded Pipeline Overview	5
3.1	Implemented pipeline stages	5
3.2	Stage-by-stage technical summary	6
4	Data Description and Data Dictionary	6
4.1	Data assets and canonical dataset facts	6
4.2	Grouped longitudinal structure	7
4.3	Key variables and data dictionary	7
4.4	Missingness and structural sparsity	9
5	Feature Engineering	10
5.1	Image-feature extraction actually implemented in code	10
5.2	Feature families	11

5.3	Stage-specific feature organization and cleanup	11
5.4	Final hidden-damage feature set	14
6	Technical Challenges and How They Were Handled	15
7	Split Strategies and Leakage Control	15
7.1	Why random row-wise splitting is invalid	15
7.2	Implemented split regimes	16
7.3	What each split regime tests	16
8	Modeling Methodology	16
8.1	Candidate model families and configuration	16
8.2	Surface stage	17
8.3	Hidden-damage stage	17
8.4	Nuance: final selected model versus per-strategy best model	18
8.5	Degradation stage	18
8.6	Proxy-RUL stage	18
9	Exploratory Analysis and Diagnostics	19
9.1	Outliers and distributions	19
9.2	Temporal corrosion progression	21
10	Results and Interpretation	22
10.1	Strong, weak, and exploratory result categories	22
10.2	Surface results	22
10.3	Hidden-damage results	22
10.4	What the feature-target correlations imply	24
11	What the Model Can and Cannot Learn	24
11.1	What is directly visible in the images	24
11.2	What may be indirectly inferable	25
11.3	What is likely not reliably recoverable here	25
12	Degradation Modelling and Proxy-RUL	25
12.1	Degradation fitting results	25
12.2	Why the degradation stage is descriptive rather than predictive	26
12.3	Proxy-threshold status	26

13 Traceability to the Original Thesis and Documentation	27
13.1 What appears to be reproduced	27
13.2 What is only approximated	27
13.3 What could not be recovered with confidence	27
14 Limitations	28
15 Conclusion	28
16 Future Work	29
A Meeting-Defense Questions and Concise Answers	29

1 Introduction

This repository implements a scientifically conservative baseline for corrosion analysis in re-inforced specimens observed repeatedly over time. The central practical question is whether visible corrosion measured from images can support a downstream chain of inference toward hidden structural degradation and an exploratory notion of residual life. The repository does not treat that question as solved. Instead, it implements a staged system that exposes which parts are supported by the data and which parts remain weak.

The key theme across the implementation is restraint. The code avoids direct end-to-end image-to-RUL modelling, uses grouped evaluation to prevent specimen leakage, begins with interpretable engineered features rather than deep models, and adds a substantial diagnostics layer so that apparent successes can be separated from tautologies, shortcuts, or unstable results.

2 Problem Framing

2.1 What problem the implemented system actually solves

The implemented system solves the following staged problem:

surface corrosion progression → hidden damage estimation → degradation modelling →
exploratory threshold-based proxy-RUL status

This wording is important. The repository does **not** implement direct supervised image-to-RUL prediction, because the dataset does not contain dense remaining-life labels. The only directly supervised image-level tasks are the surface-corrosion targets that already exist in the workbook. The hidden-damage stage is only weakly supervised because its targets exist for one terminal row per specimen. The degradation and proxy-RUL stages are downstream constructs built on predicted hidden damage.

2.2 What is directly supervised and what is not

- **Directly supervised at row level:** `surface_total_rust_pct` and `peak_rust_pct`.
- **Directly supervised only on a sparse subset:** `wire_area_loss_frac` and `ultimate_load_kn`, each available for 48 terminal rows.
- **Not directly supervised:** degradation parameters, threshold crossing time, and any true remaining useful life quantity.

2.3 Why direct image-to-RUL is not supported

The repository evidence shows that direct image-to-RUL learning is unsupported for three concrete reasons:

- there is no per-image or per-timepoint residual-life label in the aligned master table;
- structural targets are terminal-only and therefore do not provide a dense degradation trajectory;

- the implemented proxy-RUL stage uses threshold status on predicted hidden damage, which is fundamentally different from supervised RUL.

This is consistent with the dataset policy encoded in `configs/dataset.yaml` and with the implemented stage separation in the training scripts.

3 Repository-Grounded Pipeline Overview

3.1 Implemented pipeline stages

The pipeline is orchestrated through top-level scripts and supporting modules under `src/corrosion_proxy_rul/`. The major implemented stages are:

1. **Audit and validation:** `run_audit_validation.py`
2. **EDA:** `run_eda.py`
3. **Image-feature extraction:** `run_extract_image_features.py`
4. **Surface modelling:** `train_surface_models.py`
5. **Hidden-damage modelling:** `train_hidden_damage_models.py`
6. **Degradation modelling:** `train_degradation_models.py`
7. **Proxy-RUL generation:** `train_rul_proxy_models.py`
8. **Diagnostics and visualization:** `run_diagnostics_visualizations.py`

3.2 Stage-by-stage technical summary

Table 1: Repository-grounded pipeline summary.

Stage	Main input	Main output	Purpose and important design choices
Audit and validation	workbook, image directory, specimen mapping	outputs/data/master_table.csv	Pure CSVs, align workbook rows to readable images, enforce deterministic specimen mapping, log orphan or unreadable files, and create leakage-safe split manifests.
Image features	aligned image paths	outputs/features/image_features.csv	Consistent interpretable features from PNG images using thesis-aligned RGB thresholds and strip-wise summaries.
Feature engineering	master table + image features	outputs/data/full_feature_table.csv	Merge table data and image features, assign feature families, and apply stage-specific cleanup.
Surface modelling	full feature table	outputs/models/surface_damage.pkl	Benchmark visible-corrosion targets using grouped evaluation; one target is explicitly marked in code as a label-reconstruction sanity check.
Hidden-damage modelling	terminal structural subset	outputs/models/hidden_damage.pkl	Damage/feature-set ablations, target transforms, and model families using robustness-oriented selection rather than a single split.
Degradation modelling	full feature table + selected wire-area-loss model	outputs/models/degradation.pkl	Predict hidden damage for all rows, enforce monotone proxy trajectories, fit candidate curve families, and save specimen-level fits.
Proxy-RUL	degradation trajectories	outputs/models/proxy_rul.pkl	Convert hidden-damage trajectories into threshold-status outputs under fixed thresholds and a 365-day horizon.
Diagnostics	all prior outputs	outputs/diagnostics/	Audit feature behavior, missingness, collinearity, benchmark collapse, degradation behavior, and proxy-threshold status.

4 Data Description and Data Dictionary

4.1 Data assets and canonical dataset facts

The repository uses four primary external inputs:

- workbook: Data/Images_Dataset_A-Z.xlsx
- images: Data/Images_dataset/
- thesis PDF: Documentation/Thesis/s313940_Tesi_conv_msc_thesis_final_md_al_amin_hossain_s31

- conference paper PDF: [Documentation/Conferences/ARTISTE_2025___Conference_paper__10pg___A1_](#)

The verified dataset facts, as saved by the audit stage and confirmed by downstream outputs, are given in Table 2.

Table 2: Verified dataset facts after alignment and validation.

Item	Value
PNG files scanned	792
Aligned usable observations	791
Unique specimens	48
Grouped split key	<code>specimen_id</code>
Rows with <code>wire_area_loss_frac</code>	48
Rows with <code>ultimate_load_kn</code>	48
Structural-label fraction of aligned dataset	6.1%
Orphan or unreadable image	<code>E01-20240508-17W.png</code>
Campaign 1 terminal horizon	week 28 / 196 days
Campaign 2 terminal horizon	week 36 / 252 days

4.2 Grouped longitudinal structure

Each specimen appears multiple times across weeks. The aligned table stores both `week` and `ageing_days`, with `ageing_days` used as the canonical temporal variable in most downstream stages. The repeated-measures nature of the data is why row-wise random splitting is invalid: a random split would place different observations from the same specimen into both training and test sets.

4.3 Key variables and data dictionary

Table 3 documents the main columns used by the implementation. This is intentionally a focused dictionary rather than a dump of all columns.

Table 3: Key variables in the aligned master table.

Column	Meaning	Unit	Type	Role	Notes
<code>sample_name</code>	workbook observation identifier	none	string	identifier	Parsed as <code>specimen-date-week</code> .
<code>specimen_id</code>	specimen identifier from workbook	none	string	group key	Primary grouping unit for all leakage-safe splits.
<code>observation_id</code>	normalized observation identifier	none	string	identifier	Derived deterministic row key.
<code>specimen_id_from_name</code>	specimen identifier parsed from filename logic	none	string	validation field	Used to detect naming inconsistencies.

Column	Meaning	Unit	Type	Role	Notes
calendar_date	observation date	date	date-like string	metadata	Derived from sample_name.
week	exposure week encoded in name	week	integer	temporal metadata	Parsed from sample_name.
ageing_days	exposure duration	day	integer	temporal feature	Central time variable used in modelling.
campaign_id	campaign assignment	categorical	string	metadata / confounder	Reconstructed from configs/specimen_mapping.yaml
series_id	specimen-series grouping	categorical	string	metadata	Derived from naming logic.
split_group_treatment	deterministic treatment split group	categorical	string	metadata	Used for leave-one-treatment-out.
treatment_protocol	constructed treatment protocol	categorical	string	metadata	More detailed than raw workbook treatment.
treatment_coarse	coarse treatment indicator	boolean / categorical	mixed	metadata	Simplified treatment field used in reports and ablations.
treatment_label_compact	compact treatment label	categorical	string	metadata	Derived from thesis-based mapping.
n_steel_mesh	number of steel meshes	count	integer	metadata	Derived from specimen mapping; strongly correlated with campaign/time.
nacl_pct	NaCl concentration	%	float	metadata	Derived from mapping; also strongly campaign-linked.
cover_mm	concrete cover thickness	mm	float	metadata	Derived from raw workbook field.
surface_total_rust_pct	strip rust fraction on surface	fraction-like percent field	float	dense target	Available on all aligned rows. Numerically almost identical to img_rust_area_ratio_pct.
peak_rust_pct	maximum strip-wise local rust level	fraction-like percent field	float	dense target	Available on all aligned rows. More informative than surface_total_rust_pct for non-tautological surface benchmarking.
peak_rust_location	location of peak visible rust	cm	float	label / diagnostic variable	Supplied in workbook, not modelled as a primary target.
wire_area_loss_raw	raw workbook structural field	mislabeled as %	float	raw source	Raw numbers are in [0, 1] despite the header implying percent.

Column	Meaning	Unit	Type	Role	Notes
wire_area_loss_frac	normalized terminal area-loss target	fraction	float	sparse target	Canonical structural target used in modelling.
wire_area_loss_pct	scaled percentage version of area loss	%	float	derived field	Derived from <code>wire_area_loss_frac</code> .
ultimate_load_kn	measured ultimate load	kN	float	sparse target	Available only for terminal rows.
terminal_week	campaign-specific terminal week	week	integer	metadata	Derived from specimen mapping.
terminal_days	campaign-specific terminal duration	day	integer	metadata	Derived from terminal week.
has_structural_label	structural-label availability flag	none	boolean	diagnostic flag	True for 48 rows.
is_terminal_structural_row	terminal row structural row flag	none	boolean	diagnostic flag	Also true for 48 rows.
image_filename	linked PNG filename	none	string	identifier	Used in alignment.
image_readable	readability check result	none	boolean	diagnostic flag	False for the orphan unreadable image that is excluded from the aligned table.

4.4 Missingness and structural sparsity

The missingness pattern is not uniform. Dense surface targets are present for all aligned rows, while the structural targets exist only at the terminal rows. Figure 1 and Figure 2 are therefore central rather than decorative.

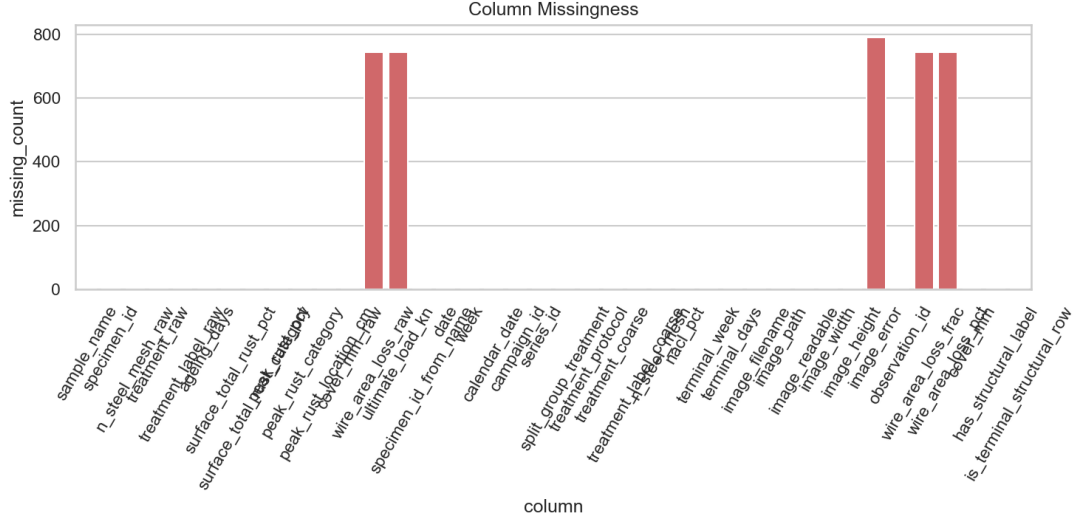


Figure 1: Column-wise missingness summary from the aligned dataset. This is strong evidence because it directly explains why the project cannot be framed as a fully supervised image-to-structure time-series problem.

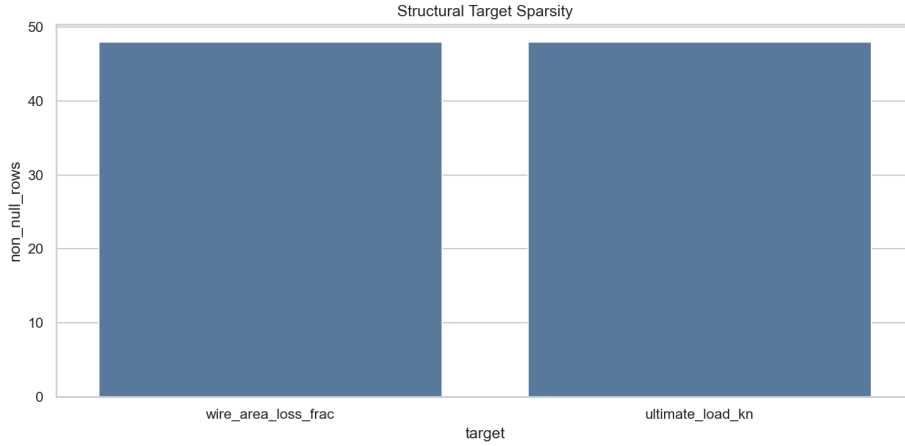


Figure 2: Structural-target sparsity. Only 48 aligned rows carry the terminal structural endpoints `wire_area_loss_frac` and `ultimate_load_kn`. This is strong evidence for the central supervision bottleneck.

5 Feature Engineering

5.1 Image-feature extraction actually implemented in code

The image-feature layer is implemented in `src/corrosion_proxy_rul/image_features.py` and configured through `configs/features.yaml`. The repository explicitly records that the exact original GIMP white-balance and cropping pipeline was **not** fully recovered. The implementation therefore uses the provided PNG files as analysis images and applies deterministic feature extraction only.

The threshold configuration is:

- rust mask RGB lower/upper: `[25, 0, 0]` to `[255, 100, 80]`

- black mask RGB lower/upper: [0, 0, 0] to [5, 5, 0]
- gray mask RGB lower/upper: [60, 70, 35] to [120, 105, 95]

The strip-wise geometry is also explicit in configuration: physical length 24.0 cm, strip width 1.0 cm, and strip step 0.5 cm.

5.2 Feature families

Table 4 groups the implemented image-derived features by physical interpretation.

Table 4: Implemented image-feature families.

Family	Representative features	Physical meaning and known issues
Mask features	img_rust_area_ratio_pct, img_black_mask_ratio_pct, img_gray_mask_ratio_pct	Area fractions under fixed RGB thresholds. These are highly interpretable but sensitive to preprocessing mismatch and threshold choice.
Brightness and color	img_brightness_mean, img_brightness_std, img_r_mean, img_g_std, img_b_std	Summaries of intensity and channel spread. Physically related to surface darkness and rust coloration, but strongly redundant.
Histogram	img_gray_hist_bin_0 ... img_gray_hist_bin_7	Distribution of grayscale intensity. Captures broad tonal shifts; moderate interpretability; high collinearity across bins is expected.
Texture	img_glcm_contrast, img_glcm_energy, img_glcm_homogeneity, img_lbp_mean, img_lbp_std	Surface roughness and local textural regularity. Potentially useful for corrosion texture but weaker and more preprocessing-sensitive than area features.
Morphology	img_rust_blob_count, img_rust_blob_largest_ratio_pct, img_rust_blob_mean_ratio_pct, img_rust_blob_eccentricity_mean	Connected rust-region geometry. Useful for roughness and clustering, but some features became near-constant on the hidden-damage dataset.
Spatial strip features	img_strip_rust_mean_pct, img_strip_rust_max_pct, img_strip_rust_std_pct, img_strip_peak_location_cm, img_rust_center_of_mass_cm	Distribution of corrosion along specimen length. High interpretability and especially relevant to peak_rust_pct. Several strip summaries are strongly redundant with global rust area.

5.3 Stage-specific feature organization and cleanup

The diagnostics layer shows that the initial feature space was highly redundant. The generated inventory `outputs/diagnostics/tables/feature_inventory_by_stage.csv` records:

- 49 included modelling features for the surface stage,
- only 8 included features for the final hidden-damage stage,
- 13 near-constant feature-stage rows,
- 1 exact duplicate feature pair,
- 36 highly collinear feature pairs.

The explicit fixed exclusions in code are:

- `img_strip_count`: constant under the chosen strip geometry,
- `img_contrast`: exact duplicate of `img_brightness_std`,
- `img_edge_to_center_rust_ratio`: unstable on low-rust images.

For hidden-damage modelling, additional cleanup removes near-constant variables when the most frequent value exceeds 0.95 of the subset and removes features with absolute Spearman correlation of at least 0.98 to a retained reference feature.

Figure 3, Figure 4, and Figure 5 summarize the most important diagnostic patterns.

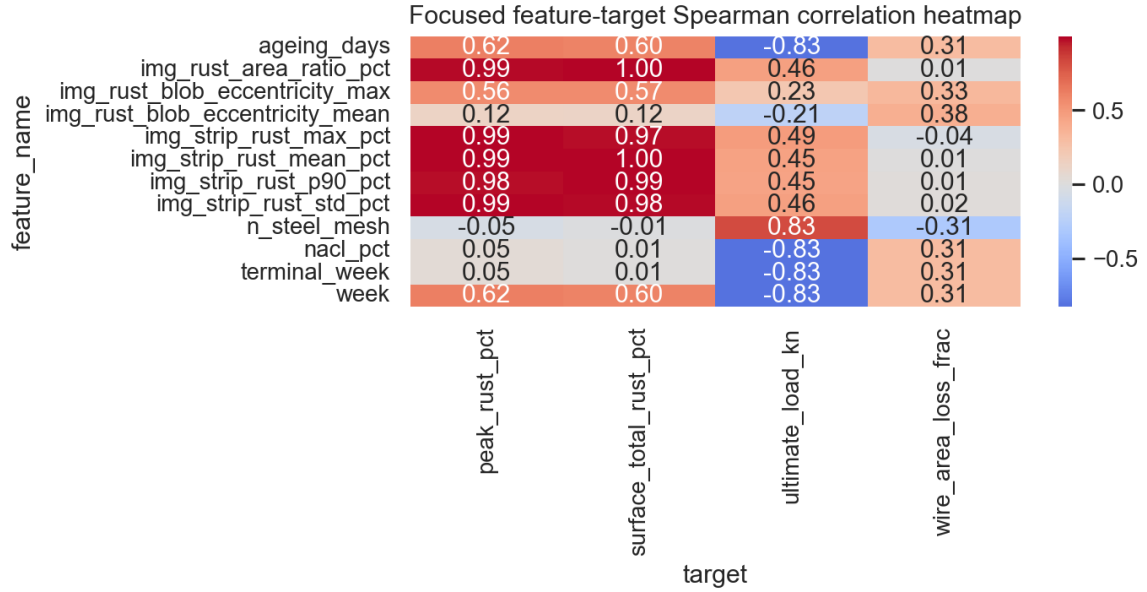


Figure 3: Feature-target Spearman correlation heatmap. This is strong evidence for two central conclusions: first, `surface_total_rust_pct` is nearly identical to `img_rust_area_ratio_pct`; second, the structural targets are much more weakly and inconsistently related to image features.

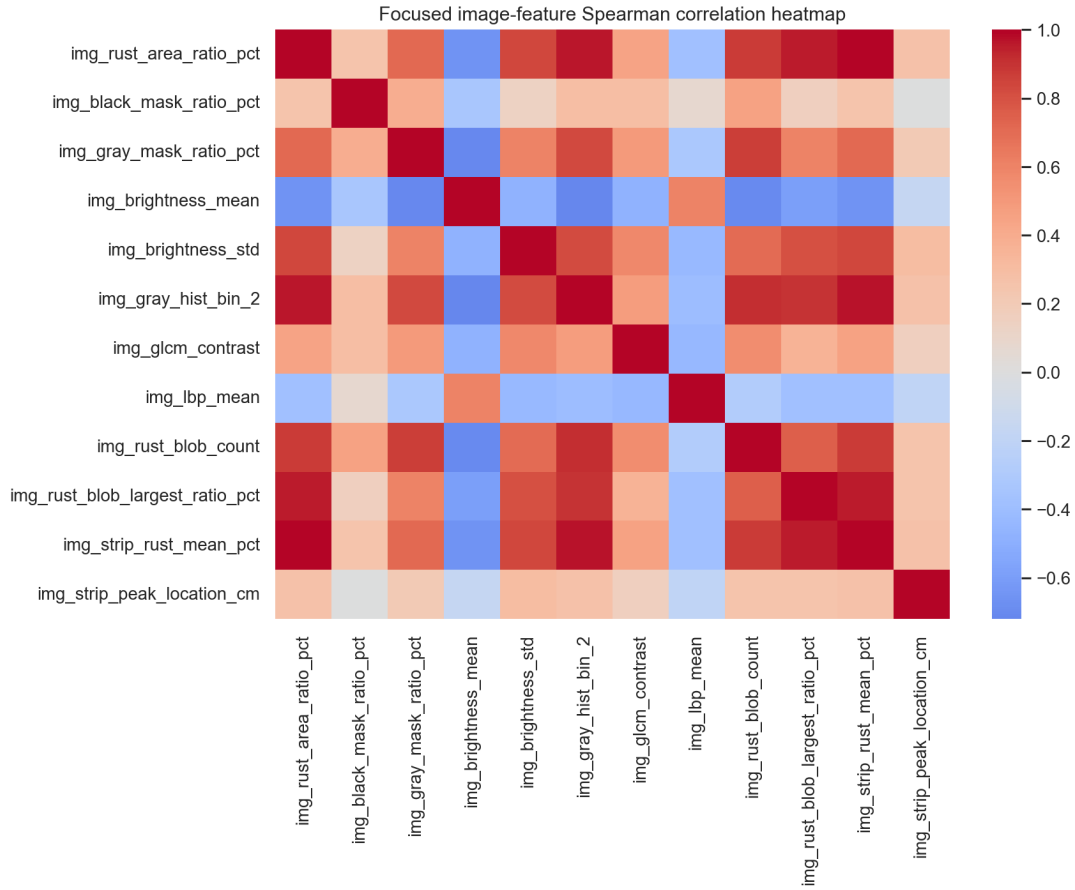


Figure 4: Focused correlation heatmap for image-derived features. This is moderate-to-strong evidence that the visible-corrosion feature space is highly structured but also substantially redundant.

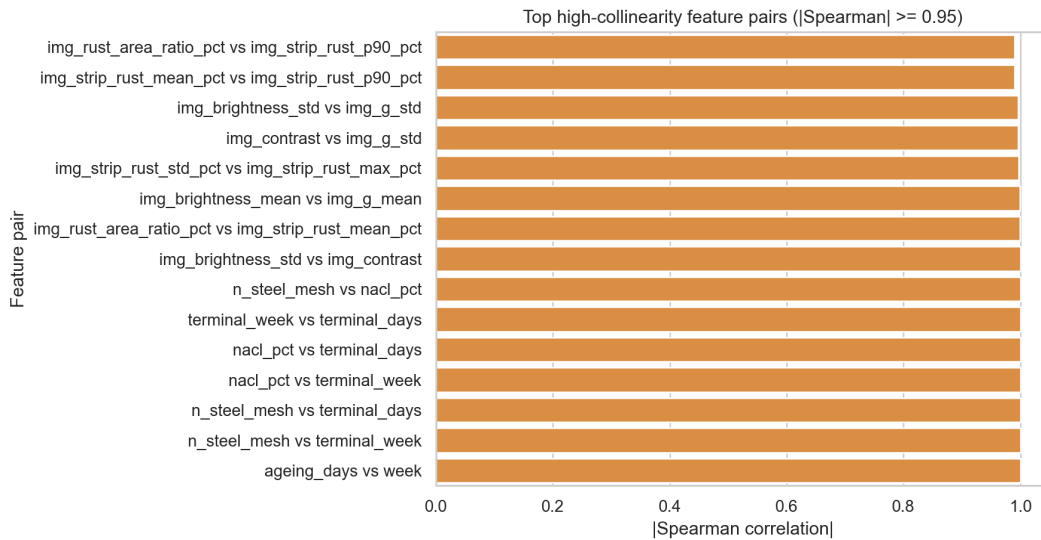


Figure 5: High-collinearity pairs among features retained for diagnostics. This is strong evidence supporting the cleanup stage and explaining why apparently different feature families sometimes carry nearly the same information.

5.4 Final hidden-damage feature set

After the improvement round, the final selected hidden-damage models are metadata-only and use the same eight features for both structural targets:

- ageing_days
- campaign_id
- cover_mm
- series_id
- split_group_treatment
- treatment_coarse
- treatment_label_coarse
- treatment_protocol

This is repository-verified through `outputs/models/hidden_damage/hidden_damage_selected_feature_list`. It is also a scientifically important warning sign: the most robust current structural signal is dominated by exposure and design metadata, not by the interpretable image features.

6 Technical Challenges and How They Were Handled

Table 5: Main technical challenges documented in the repository.

Challenge	How it was detected	How it was handled	Residual risk
Workbook-image misalignment	audit image scan plus workbook merge	Unreadable and orphan files were logged explicitly; aligned table uses only readable matched images.	One raw orphan image remains in the source folder.
Corrupted/orphan image E01-20240508-17W.png	outputs/audit/issue_logs.csv and image scan	Excluded from aligned dataset, issue preserved in logs.	One specimen therefore has 17 rather than 18 observations in campaign 2.
Specimen naming and treatment ambiguity	mismatch between workbook fields and thesis logic	Deterministic mapping moved to configs/specimen_mapping.yaml	Mapping still depends on thesis reconstruction rather than explicit source columns alone.
Wire-area-loss unit inconsistency	raw field values contradicted % header	Canonical target stored as <code>wire_area_loss_frac</code> ; percentage version derived explicitly.	Raw source semantics remain inconsistent.
Repeated-measures leakage risk	dataset structure itself	All evaluation implemented at specimen level; split summaries verify zero specimen leakage.	No residual row-wise leakage is evident, but campaign dependence remains.
Structural-label sparsity	missingness tables and terminal subset size	Structural stage limited to 48 rows; conclusions written conservatively.	The sparse supervision ceiling remains the dominant limitation.
Feature redundancy and duplication	diagnostics tables and heatmaps	Constant, duplicate, unstable, near-constant, and highly collinear features excluded or flagged.	Remaining metadata still embeds campaign shortcuts.
Campaign confounding	leave-one-campaign-out collapse and ablation tables	Added metadata-only, image-only, and image-time ablations; final model selection weighted robustness heavily.	Current best structural models remain metadata-only.
Degradation fit instability	<code>degradation_fit_candidates</code>	Failed candidate fits are kept with failure flags; best-fit selection ignores failed fits.	21 gompertz fits failed to converge.
Proxy-RUL interpretability	threshold summaries showed no future crossings	Reframed from pseudo-RUL days to threshold-status reporting.	Still exploratory because no forward crossing within horizon is observed.

7 Split Strategies and Leakage Control

7.1 Why random row-wise splitting is invalid

Random row-wise splitting would leak specimen identity because each specimen contributes multiple observations across time. This would allow a model to see the same specimen in training and testing under slightly different weeks, which is scientifically invalid for progression

modelling.

7.2 Implemented split regimes

The split logic is implemented in `src/corrosion_proxy_rul/splits.py`. The repository uses three regimes:

- **GroupShuffleSplit by specimen:** repeated grouped holdout for average within-dataset generalization.
- **Leave-one-treatment-out:** stress test across reconstructed treatment groups.
- **Leave-one-campaign-out:** strongest domain-shift test across campaigns.

Table 6 summarizes the actual saved split manifests.

Table 6: Saved split-manifest summary.

Strategy	Train specimens	Test specimens	Leakage
GroupShuffleSplit (5 folds)	38 per fold	10 per fold	False in all folds
Leave-one-treatment-out (9 folds)	41 to 45	3 to 7	False in all folds
Leave-one-campaign-out (2 folds)	24	24	False in both folds

7.3 What each split regime tests

- **GroupShuffleSplit** tests generalization to unseen specimens under similar global data distribution.
- **Leave-one-treatment-out** tests whether the model depends heavily on a specific treatment grouping.
- **Leave-one-campaign-out** tests whether the model survives the strongest bundled domain shift in the repository, including campaign-linked duration, mesh configuration, and salt concentration differences.

Performance degradation under stricter splits is therefore not just a nuisance. It is a scientific signal about shortcut learning, domain shift, or lack of transferable structure-target mapping.

8 Modeling Methodology

8.1 Candidate model families and configuration

The candidate regressors are fixed in `configs/modeling.yaml`: Random Forest, Gradient Boosting, XGBoost, and CatBoost. Table 7 lists the main configured hyperparameters.

Table 7: Main model configuration from `configs/modeling.yaml`.

Model	Configured hyperparameters
Random Forest	<code>n_estimators=300, max_depth=8, min_samples_leaf=2, random_state=42.</code>
Gradient Boosting	<code>n_estimators=250, learning_rate=0.05, max_depth=3, random_state=42.</code>
XGBoost	<code>n_estimators=300, learning_rate=0.05, max_depth=4, subsample=0.9, colsample_bytree=0.9, reg_lambda=1.0.</code>
CatBoost	<code>iterations=300, learning_rate=0.05, depth=6, loss_function=RMSE, random_seed=42, verbose=False.</code>

All modelling stages compute MAE, RMSE, R^2 , and Spearman correlation. Hidden-damage experiments additionally compare target transforms `none` and `sqrt` where configured.

8.2 Surface stage

The surface stage models two targets:

- `surface_total_rust_pct`
- `peak_rust_pct`

The code in `src/corrosion_proxy_rul/models_surface.py` explicitly labels `surface_total_rust_pct` as `label_reconstruction_sanity_check`. This is justified by the measured near-identity between the label and the engineered rust-area feature:

- absolute-difference mean between `surface_total_rust_pct` and `img_rust_area_ratio_pct`: 2.28×10^{-4}
- maximum absolute difference: 4.999×10^{-4}
- Spearman correlation: 0.9996

Therefore, only `peak_rust_pct` should be treated as a more meaningful surface benchmark.

8.3 Hidden-damage stage

The hidden-damage stage is implemented in `src/corrosion_proxy_rul/models_hidden_damage.py`. It uses the 48-row terminal structural subset and compares four feature-set definitions:

- `all_cleaned`: 42 features
- `image_only`: 34 features
- `image_time_only`: 35 features
- `metadata_only`: 8 features

For `wire_area_loss_frac`, both `none` and `sqrt` target transforms are evaluated. For `ultimate_load_kn`, only the untransformed target is used.

Final model selection is not based on one split. It uses a weighted robustness score recorded in configuration:

- GroupShuffle MAE weight: 0.20

- Leave-one-treatment-out MAE weight: 0.25
- Leave-one-campaign-out MAE weight: 0.45
- GroupShuffle Spearman weight: 0.10

This weighting intentionally prioritizes cross-campaign robustness over a single easier split.

8.4 Nuance: final selected model versus per-strategy best model

The repository contains two different but both legitimate benchmark summaries:

- `outputs/models/hidden_damage/best_models.csv`: final robustness-selected model for each structural target.
- `outputs/diagnostics/tables/benchmark_best_model_robustness.csv`: best observed model for each target and strategy in the diagnostics layer.

For `wire_area_loss_frac`, these differ. The final deployment-selected model is CatBoost with the `metadata_only` feature set, but the diagnostics table shows Random Forest as the best *per-strategy* snapshot under some strategies. This is not an error; it reflects the difference between robustness-weighted final selection and strategy-wise local minima.

8.5 Degradation stage

The degradation stage is implemented in `src/corrosion_proxy_rul/models_degradation.py`. Its inputs are:

- the full feature table,
- the saved `wire_area_loss_frac` group-shuffle best-model artifact,
- configured candidate curve families.

The code first predicts a hidden-damage proxy for every aligned row, then applies a specimen-wise monotone projection before fitting candidate trajectory families:

- linear
- log-time
- logistic
- gompertz
- monotone isotonic

Model selection uses a BIC-like criterion with family complexity penalties from configuration. Candidate-fit logs show 240 family-specimen attempts in total, of which 21 failed, all from the gompertz family due to non-convergence.

8.6 Proxy-RUL stage

The proxy stage is implemented in `src/corrosion_proxy_rul/models_rul_proxy.py`. It does not learn RUL. Instead, it evaluates hidden-damage trajectories against fixed thresholds:

- 0.20
- 0.30
- 0.40

under a 365-day projection horizon. The output is threshold status: crossed by baseline, crossed during observed history, future crossing within horizon, or right-censored.

9 Exploratory Analysis and Diagnostics

9.1 Outliers and distributions

The diagnostics layer deliberately preserves full-range statistics while also providing readability-focused views. Figure 6 is a trimmed $p99$ view for skewed variables; the repository keeps the full-range summaries separately in CSV tables.

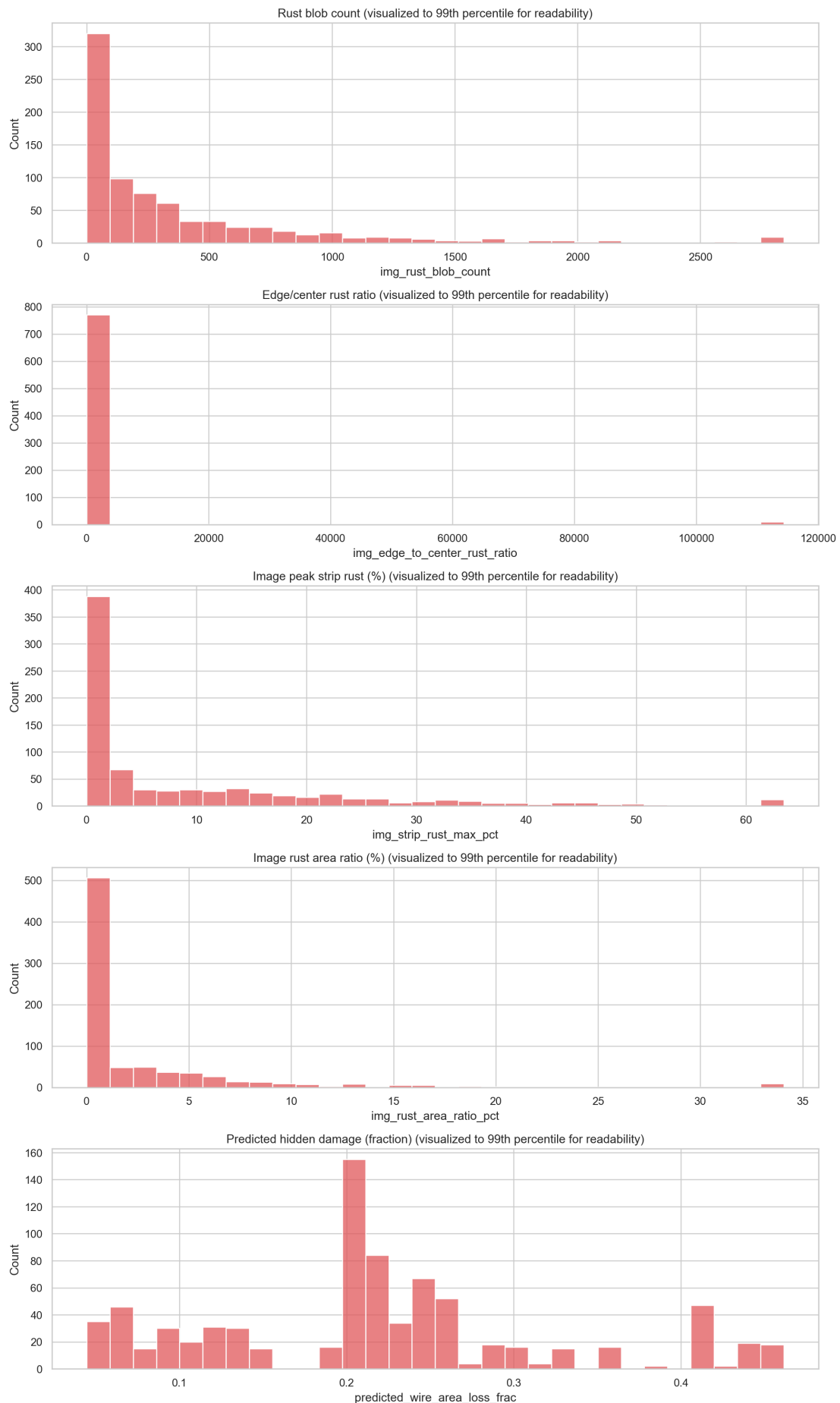


Figure 6: Trimmed p_{99} histogram view for skewed variables. This is moderate evidence: it improves readability without discarding true extremes, but **20** should always be interpreted alongside the saved full-range summary tables.

9.2 Temporal corrosion progression

The temporal progression plots are among the strongest outputs in the entire repository because they use dense labels, repeated observations, and grouped specimen structure directly rather than relying on sparse structural endpoints.

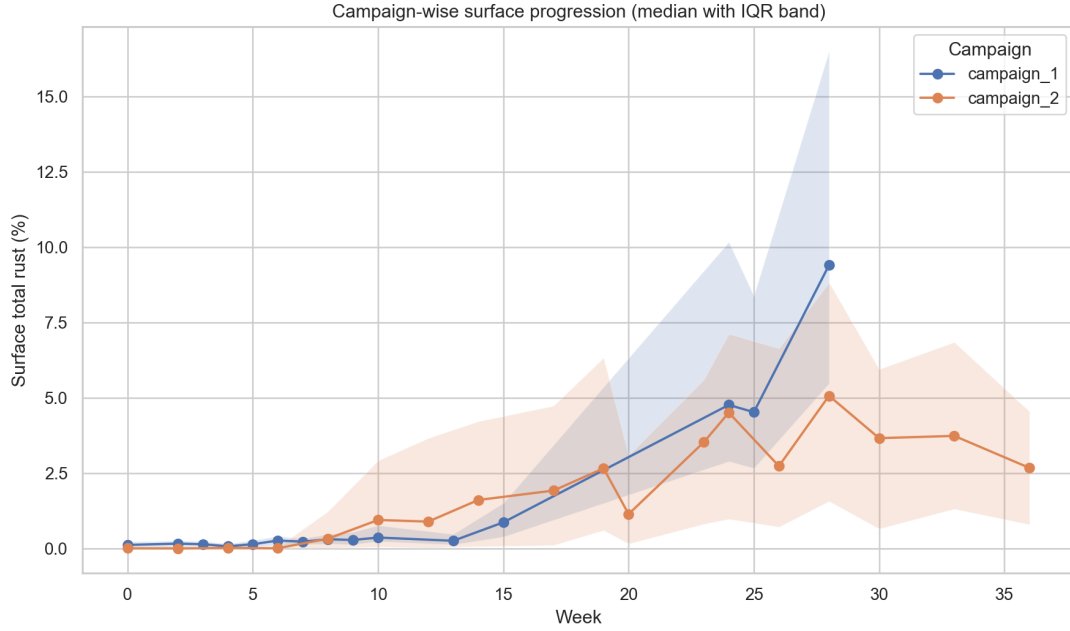


Figure 7: Campaign-wise median and interquartile progression of visible corrosion over time. This is strong evidence that visible corrosion progression is measurable and systematically different across campaign conditions.

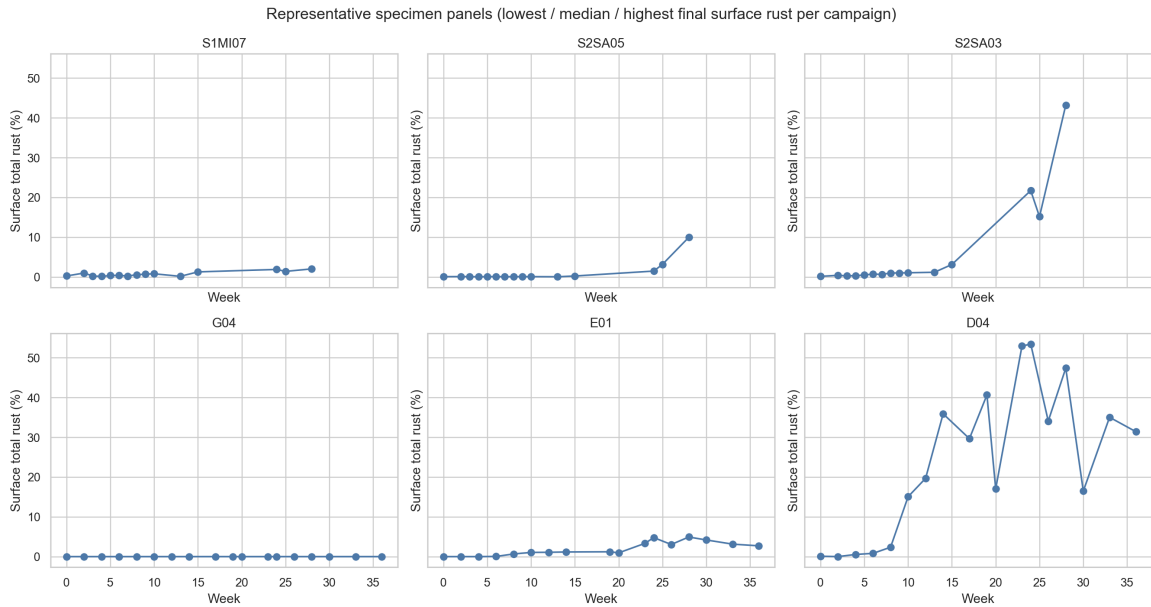


Figure 8: Representative specimen trajectories for visible corrosion. This is strong evidence that within-specimen corrosion progression is heterogeneous but clearly time-structured.

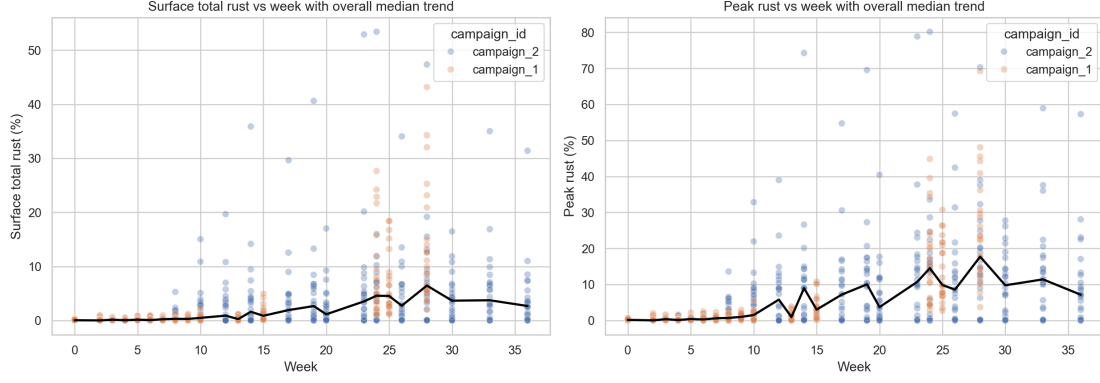


Figure 9: Scatter-plus-trend view of visible corrosion against time. This is moderate-to-strong evidence because it summarizes the global temporal relation while still showing dispersion.

10 Results and Interpretation

10.1 Strong, weak, and exploratory result categories

The repository outputs support three different evidence levels:

- **Strong:** data audit, leakage control, temporal surface progression, feature redundancy diagnostics, and the conclusion that `surface_total_rust_pct` is reconstructive.
- **Weak to moderate:** `peak_rust_pct` benchmarking and some grouped structural performance for `ultimate_load_kn`.
- **Exploratory:** hidden-damage generalization across campaigns, degradation family interpretation, and threshold-based proxy-RUL status.

10.2 Surface results

The surface benchmark results are easy to misread, so the repository framing matters:

- `surface_total_rust_pct`: Gradient Boosting, GroupShuffle MAE = 0.1773, Spearman = 0.9988. This is **not** strong scientific predictive evidence because the target is almost identical to the engineered rust-area feature.
- `peak_rust_pct`: Random Forest, GroupShuffle MAE = 1.0968, Spearman = 0.9923, Leave-one-campaign-out MAE = 1.0577, Spearman = 0.9769. This is a more meaningful visible-corrosion benchmark because it depends on spatial strip structure rather than a near-identity global mask.

10.3 Hidden-damage results

The final selected structural models are summarized in Table 8.

Table 8: Final robustness-selected hidden-damage models from outputs/models/hidden_damage/best_models.csv.

Target	Selected model	Feature set	Features	GS MAE	LOTO MAE	LOCO MAE
wire_area_loss_frac	CatBoost	metadata-only	8	0.1238	0.1161	0.1208
ultimate_load_kn	Random Forest	metadata-only	8	0.1744	0.2034	0.5642

These results require careful interpretation:

- **ultimate_load_kn:** the grouped split result is usable as a weak-to-moderate signal, but leave-one-campaign-out deteriorates substantially. The selected model remains metadata-only, indicating that the strongest currently usable signal is campaign-linked exposure/design information.
- **wire_area_loss_frac:** the target remains weakly modelled. Spearman is low on grouped splits and negative under leave-one-campaign-out for the selected final model. R^2 is negative across strategies for the selected model family. This target should not be presented as reliably inferable from the current dataset.

Figure 10 and Figure 11 are the most important benchmark visuals.

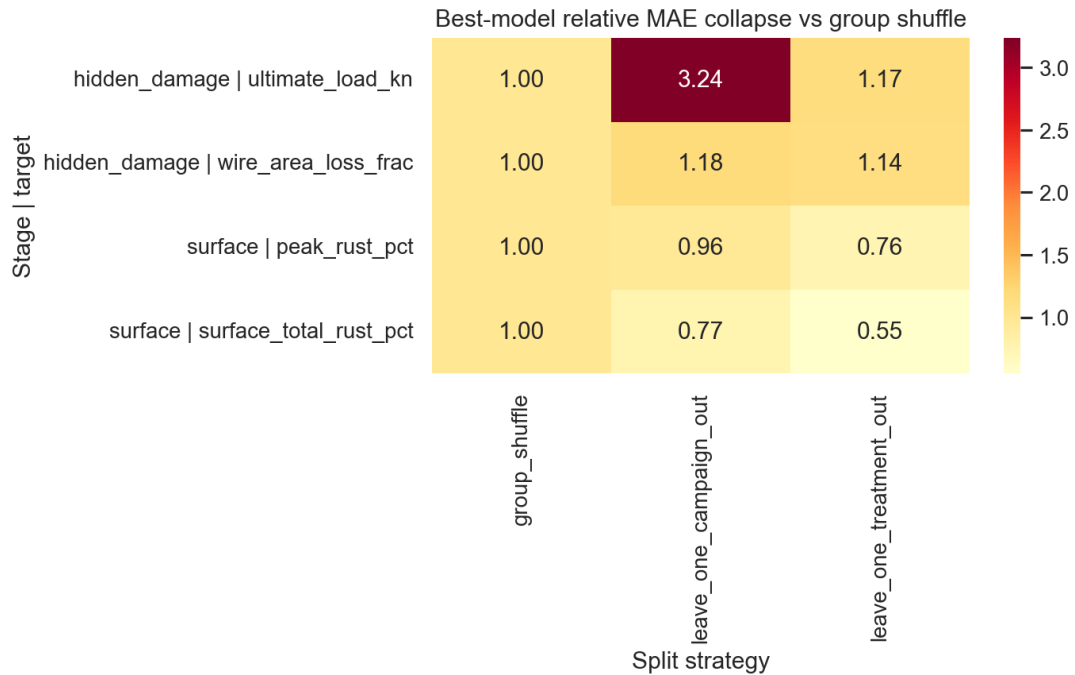


Figure 10: Relative MAE collapse across split strategies. This is strong evidence because it directly shows where apparent performance fails under stricter evaluation. Structural targets degrade much more than surface targets.

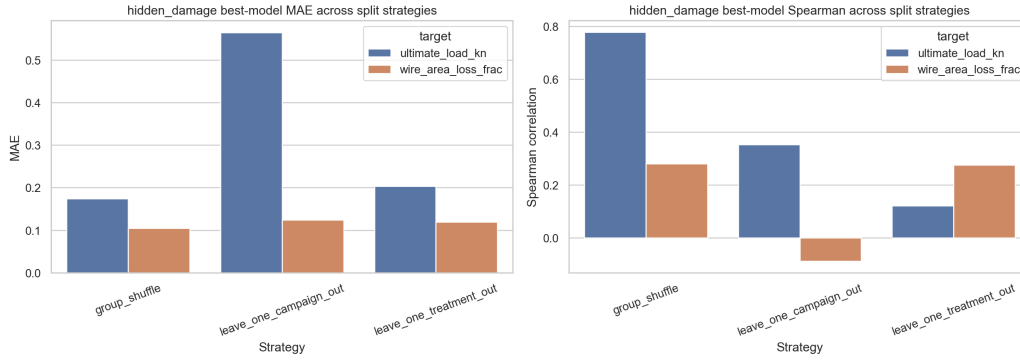


Figure 11: Robustness comparison for the best hidden-damage models across split strategies. This is strong evidence that the structural stage remains fragile, particularly under leave-one-campaign-out evaluation.

10.4 What the feature-target correlations imply

The correlation diagnostics sharpen the modelling interpretation:

- For `surface_total_rust_pct`, the dominant signal is `img_rust_area_ratio_pct`, which makes the task reconstructive.
- For `peak_rust_pct`, the strongest features are spatial strip summaries, especially `img_strip_rust_std_pct` and `img_strip_rust_max_pct`. This is consistent with what the target measures physically.
- For `wire_area_loss_frac`, the strongest univariate image relation is only about 0.382 in absolute Spearman terms, and many image features are near zero. This is weak evidence for recoverable hidden-damage signal from images alone.
- For `ultimate_load_kn`, the strongest correlates are time and campaign-linked metadata rather than image features, which explains why metadata-only models are favored by the robustness selector.

11 What the Model Can and Cannot Learn

11.1 What is directly visible in the images

The implemented feature set can directly quantify:

- the visible fraction of rust-like pixels under fixed thresholds,
- where corrosion appears concentrated along the specimen,
- how patchy or spatially dispersed rust regions are,
- broad changes in brightness, color spread, and texture.

These are legitimate surface-corrosion descriptors and the temporal plots show that they carry real progression information.

11.2 What may be indirectly inferable

Hidden structural degradation may be indirectly inferable only to the extent that it is correlated with visible corrosion, exposure time, specimen configuration, and treatment conditions. The repository results indicate that this indirect path is weak and heavily entangled with metadata.

11.3 What is likely not reliably recoverable here

The current dataset does not support reliable recovery of:

- dense hidden structural trajectories,
- true residual life labels,
- strong campaign-robust image-only inference of terminal structural damage.

This explains why visible corrosion is learned well, why the final structural models rely on metadata, and why proxy-RUL remains exploratory.

12 Degradation Modelling and Proxy-RUL

12.1 Degradation fitting results

The current degradation summary in `outputs/models/degradation/degradation_summary.json` reports 48 fitted specimens, using the saved `wire_area_loss_frac` model artifact as the upstream hidden-damage predictor. The current best-family counts are:

- linear: 27 specimens
- monotone isotonic: 21 specimens

The candidate-fit table contains 240 family-specimen attempts, with 21 failures, all in the gompertz family due to non-convergence. That failure pattern should be documented rather than hidden.

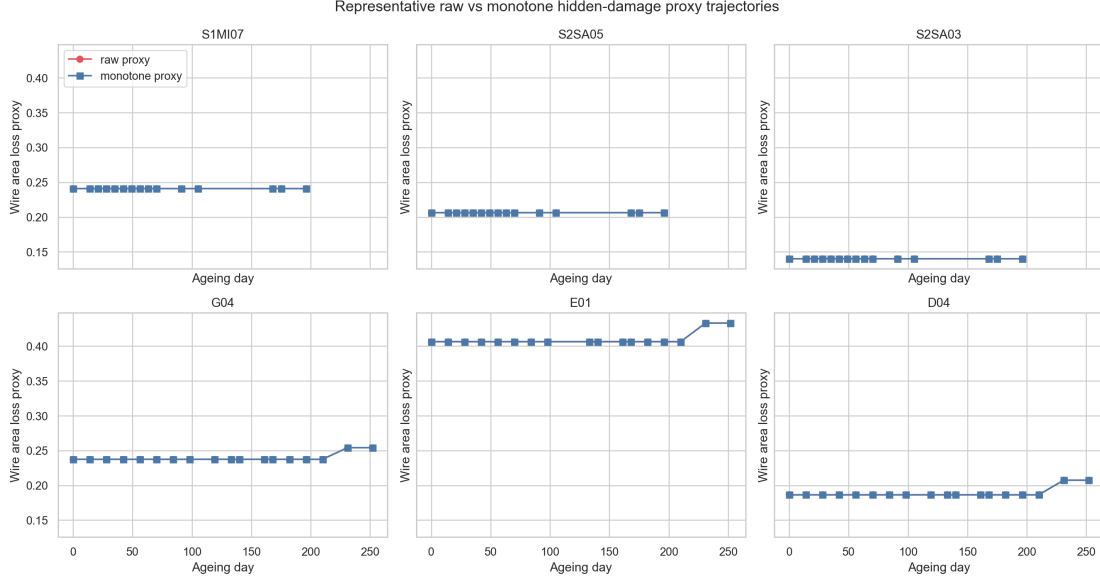


Figure 12: Representative degradation trajectories before and after the monotone projection used for fitting. This is moderate evidence for the mechanics of the implemented stage, but weak evidence for predictive structural degradation because the trajectories are built from model-predicted hidden damage rather than dense observed structural measurements.

12.2 Why the degradation stage is descriptive rather than predictive

The degradation curves are not fitted to observed wire-area-loss trajectories, because such trajectories do not exist in the dataset. They are fitted to model-predicted hidden damage propagated across all rows. That makes the stage useful for internal consistency analysis and threshold-status exploration, but not strong enough for prospective structural prediction claims.

12.3 Proxy-threshold status

The proxy stage evaluates the fitted trajectories at thresholds 0.20, 0.30, and 0.40. The current summary is:

- threshold 0.20: 32 crossed by baseline, 1 crossed during observed history, 0 future crossings within horizon, 15 right-censored
- threshold 0.30: 8 crossed by baseline, 1 crossed during observed history, 0 future crossings within horizon, 39 right-censored
- threshold 0.40: 5 crossed by baseline, 0 crossed during observed history, 0 future crossings within horizon, 43 right-censored

This means the current proxy stage provides threshold-status partitioning, not actionable forward RUL estimates.

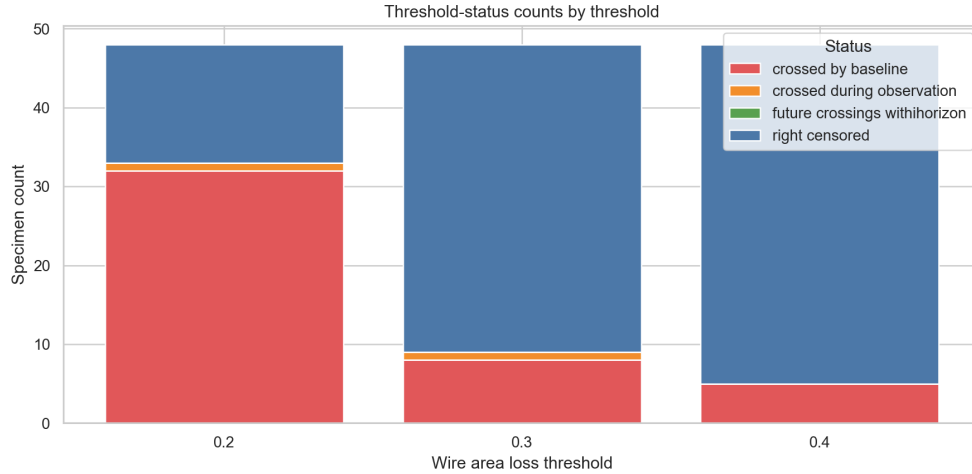


Figure 13: Proxy threshold-status counts across the three configured wire-area-loss thresholds. This is honest but weak evidence: it shows that the current proxy stage is dominated by already-crossed and right-censored cases, with no future threshold crossing inside the 365-day horizon.

13 Traceability to the Original Thesis and Documentation

13.1 What appears to be reproduced

The implementation appears to reproduce or approximate the following thesis-derived ideas:

- specimen naming logic and campaign structure,
- treatment grouping logic encoded in `configs/specimen_mapping.yaml`,
- strip-wise corrosion characterization,
- rust, black, and gray threshold-based surface quantification,
- use of surface corrosion as the first stage of a larger structural reasoning chain.

13.2 What is only approximated

The repository explicitly states in configuration that the exact GIMP white-balance and cropping workflow was not fully recovered. The code therefore treats the provided PNGs as analysis-ready inputs. That is an approximation, not an exact reproduction of the original documented workflow.

13.3 What could not be recovered with confidence

The following remain ambiguous or only partially recoverable from the repository:

- the exact raw-to-PNG preprocessing steps used before the current image set was created,
- whether the workbook surface label was generated by precisely the same thresholds as the current rust-area feature,
- how much manual intervention, if any, existed in the original thesis-side image analysis chain,

- the exact feature-set label for the wire-area-loss model referenced inside the current degradation summary JSON, which stores an empty feature-set string even though `best_models.csv` shows the final selected feature set as metadata-only.

These ambiguities do not invalidate the implementation, but they do limit the strength of any claim about exact method replication.

14 Limitations

The limitations below are not generic. They follow directly from the implemented repository and generated outputs:

- **Sample size and specimen count:** 791 rows is not tiny for surface analysis, but 48 specimens is small for robust grouped generalization and very small for structural endpoint modelling.
- **Structural target sparsity:** only 48 terminal rows carry `wire_area_loss_frac` and `ultimate_load_kn`.
- **Surface-target tautology risk:** `surface_total_rust_pct` is almost numerically identical to `img_rust_area_ratio_pct`.
- **Campaign confounding:** campaign, mesh count, salt concentration, and terminal duration are bundled together strongly enough that metadata can dominate hidden-damage prediction.
- **Metadata shortcut dependence:** the final selected hidden-damage models are metadata-only.
- **Cross-campaign fragility:** leave-one-campaign-out performance collapses for structural targets relative to grouped validation.
- **Preprocessing mismatch risk:** exact thesis preprocessing could not be reproduced, so image-derived features may not align exactly with the original analysis chain.
- **Degradation-stage identifiability:** fitted degradation curves are based on predicted hidden damage, not dense observed structural trajectories.
- **Proxy-RUL uncertainty:** thresholds are configured and deterministic, but no future crossings occur within horizon, so the stage is better described as threshold-status analysis than residual-life estimation.

15 Conclusion

The repository successfully delivers a leakage-safe, interpretable, and well-instrumented baseline pipeline for corrosion-image analysis. The implemented system is strongest where the data are strongest: audit and alignment, visible-corrosion quantification, temporal progression analysis, and feature diagnostics. The repository is also scientifically useful because it exposes negative results clearly: hidden-damage inference is weak, cross-campaign robustness is poor, and proxy-RUL is exploratory rather than deployable.

The most important technical conclusion is that the current implementation learns visible corrosion more convincingly than it learns hidden structural damage. Where structural signal is

most robust, it is largely metadata-driven. That is not a failure of the code. It is a property of the current data and supervision regime, and the report should be read accordingly.

16 Future Work

The next steps should follow directly from the observed bottlenecks rather than from generic modelling ambition:

- reduce campaign shortcut dependence further within the current interpretable framework;
- quantify what structural signal remains when the most campaign-linked metadata are ablated;
- recover any missing thesis-side preprocessing details or original strip-analysis outputs, if available;
- add uncertainty reporting around the hidden-damage predictions instead of relying only on point metrics;
- avoid treating the proxy stage as residual-life prediction unless denser structural supervision becomes available.

A Meeting-Defense Questions and Concise Answers

Why was direct RUL prediction not implemented?

Because the repository contains no dense RUL labels. Structural endpoints exist only on 48 terminal rows, so a direct supervised image-to-RUL model would not be grounded in actual labels.

Why are grouped splits mandatory?

The same specimen appears repeatedly over time. Row-wise random splits would leak specimen-specific information from training to testing.

Why does robustness drop under leave-one-campaign-out?

Because campaign bundles together duration, mesh configuration, and salt concentration. What looks predictive under grouped holdout often does not survive a full campaign shift.

Why can visible corrosion be predicted better than hidden structural degradation?

Visible corrosion is directly present in the images and densely labelled. Hidden structural degradation is only sparsely supervised at terminal rows and is not fully observable in the images.

How much of the current structural signal comes from metadata?

In the final robustness-selected models, essentially all retained features are metadata or time variables. The final selected hidden-damage feature set contains no image features.

What is the highest-risk assumption in the image pipeline?

That the provided PNGs can be treated as analysis images even though the exact original GIMP preprocessing was not fully recoverable.

Could preprocessing mismatch explain part of the structural-model weakness?

Possibly, but it cannot explain the entire problem. Structural sparsity, campaign confounding, and terminal-only supervision are stronger documented limitations.

Which feature is most interpretable?

`img_rust_area_ratio_pct` is the most direct physical descriptor of visible rust area. It is also the clearest example of a feature that is too close to a supplied surface label to be treated as a substantive predictive discovery.

What does Spearman add beyond MAE?

MAE measures absolute error magnitude; Spearman measures rank consistency. This matters because some wire-area-loss models achieve similar MAE while still failing to preserve ordering, especially under leave-one-campaign-out.

What should be shown first in a technical discussion?

The missingness and structural sparsity figures, the temporal progression figures, the feature-target correlation heatmap, and the benchmark collapse heatmap. Those four areas explain the whole project more honestly than a single “best score” table.